Homework 12

This homework is due at 11 PM on April 24, 2026.

Submission Format: Your homework submission should consist of a single PDF file that contains all of your answers (any handwritten answers should be scanned), as well as a printout of your completed Jupyter notebook(s).

1. **Formulating problems as LPs or QPs**

   Formulate the problem

   $$p_j^* \doteq \min_{\vec{x}} \; f_j(\vec{x}), \tag{1}$$

   for different functions f_j, $j = 1, \ldots, 4$, as convex **QPs** or **LPs**, or, if you cannot, explain why. In our formulations, we always use $\vec{x} \in \mathbb{R}^n$ as the variable, and assume that $A \in \mathbb{R}^{m \times n}$, $\vec{y} \in \mathbb{R}^m$. If you obtain a convex **LP** or **QP** formulation, state precisely what the variables, objective, and constraints are.

   (a) $f_1(\vec{x}) = \|A\vec{x} - \vec{y}\|_\infty + \|\vec{x}\|_1$.

   (b) $f_2(\vec{x}) = \|A\vec{x} - \vec{y}\|_2^2 + \|\vec{x}\|_1$.

   (c) $f_3(\vec{x}) = \|A\vec{x} - \vec{y}\|_2^2 - \|\vec{x}\|_1$.

   (d) $f_4(\vec{x}) = \|A\vec{x} - \vec{y}\|_2^2 + \|\vec{x}\|_1^2$.

2. A Slalom Problem

A skier must slide from left to right by going through n parallel gates of known position (x_i, y_i) and width c_i, $i = 1, \ldots, n$. The initial position (x_0, y_0) is given, as well as the final one, (x_{n+1}, y_{n+1}). Before reaching the final position, the skier must go through gate i by passing between the points $(x_i, y_i - c_i/2)$ and $(x_i, y_i + c_i/2)$ for each $i \in \{1, \ldots, n\}$.

Figure 1 is an example and does not have the right value of n nor show the true (x_i, y_i, c_i) values. Use values for (x_i, y_i, c_i) from Table 1.

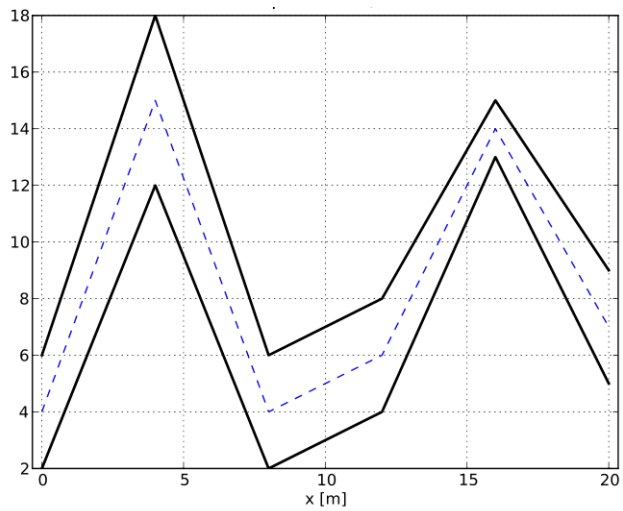


Figure 1: Slalom problem with $n = 6$ gates. The initial and final positions are fixed and not included in the figure. The skier slides from left to right. The middle path is dashed and connects the center points of gates.

Table 1: Problem data for Problem 2. Here $n = 5$.

i	x_i	y_i	c_i
0	0	4	N/A
1	4	5	3
2	8	4	2
3	12	6	2
4	16	8	1
5	20	7	2
6	24	4	N/A

(a) Given the data $\{(x_i, y_i, c_i)\}_{i=0}^{n+1}$, write an optimization problem that minimizes the total length of the path. Your answer should come in the form of an SOCP.

(b) Solve the problem numerically with the data given in Table 1. *HINT: You should be able to use packages such as* `cvxpy` *and* `numpy`*.*

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

3. Dual Norms and SOCP

Consider the problem

$$p^\star = \min_{\vec{x} \in \mathbb{R}^n} \|A\vec{x} - \vec{y}\|_1 + \mu \|\vec{x}\|_2 , \tag{2}$$

where $A \in \mathbb{R}^{m \times n}$, $\vec{y} \in \mathbb{R}^m$, and $\mu > 0$.

(a) Express this (primal) problem in standard SOCP form. *HINT: you can make the objective function linear by introducing slack variables.*

(b) Find a dual to the problem and express it in standard SOCP form. *HINT: You may find Problem 1 in Discussion 13 to be helpful. Recall that for every vector $\vec{z}$, the following dual norm equalities hold:*

$$\|\vec{z}\|_2 = \max_{\vec{u} \,:\, \|\vec{u}\|_2 \leq 1} \vec{u}^\top \vec{z}, \qquad \|\vec{z}\|_1 = \max_{\vec{u} \,:\, \|\vec{u}\|_\infty \leq 1} \vec{u}^\top \vec{z}. \tag{3}$$

Additionally, you may use the following fact: If the primal problem is expressed as $p^\star = \min_{\vec{x}} \max_{\vec{y}} L(\vec{x}, \vec{y})$ (which is $p^\star = \min_{\vec{x}} f_0(x)$ for $f_0(x) = \max_{\vec{y}} L(\vec{x}, \vec{y})$), then the dual problem can be obtained by swapping the max and min: $d^\star = \max_{\vec{y}} \min_{\vec{x}} L(\vec{x}, \vec{y})$.

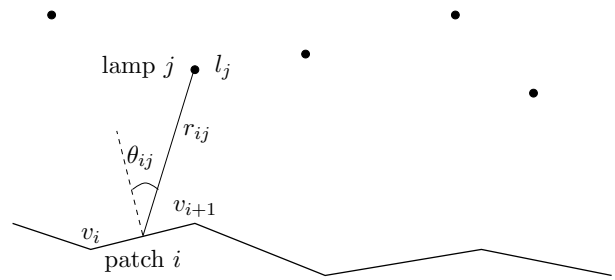
(c) Assume strong duality holds[1] and that $m = 100$ and $n = 10^6$, i.e., A is 100×10^6. Which problem would you choose to solve using a numerical solver: the primal or the dual? Justify your answer.

[1] In fact, you can show that strong duality holds using Sion's theorem, a generalization of the minimax theorem that is beyond the scope of this class.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

4. An illumination problem.

We consider an illumination system of m lamps, at positions $\vec{l_1}, \ldots, \vec{l_m} \in \mathbb{R}^2$, illuminating n flat patches.



The patches are line segments; the ith patch is given by $[\vec{v_i}, \vec{v_{i+1}}]$ where $\vec{v_1}, \ldots, \vec{v_{n+1}} \in \mathbb{R}^2$. The variables in the problem are the lamp powers $p_1, \ldots, p_m$, which can vary between 0 and 1.

The illumination at (the midpoint of) patch i is denoted I_i. We will use a simple model for the illumination:

$$I_i = \sum_{j=1}^{m} a_{ij} p_j, \quad a_{ij} = r_{ij}^{-2} \max\{\cos\theta_{ij}, 0\}, \tag{7}$$

where r_{ij} denotes the distance between lamp j and the midpoint of patch i, and θ_{ij} denotes the angle between the upward normal of patch i and the vector from the midpoint of patch i to lamp j, as shown in the figure. This model takes into account "self-shading" (i.e., the fact that a patch is illuminated only by lamps in the halfspace it faces) but not shading of one patch caused by another. Of course we could use a more complex illumination model, including shading and even reflections. This just changes the matrix relating the lamp powers to the patch illumination levels.

The problem is to determine lamp powers that make the illumination levels close to a given desired illumination level I_{des}, subject to the power limits $0 \le p_i \le 1$.

(a) Suppose we use the maximum deviation

$$\phi(p) = \max_{k=1,\ldots,n} |I_k - I_{\text{des}}|$$

as a measure for the deviation from the desired illumination level. Formulate the illumination problem using this criterion as a linear programming problem.

(b) There are several suboptimal approaches based on weighted least-squares. We consider two examples.

i *Saturated least-squares.* We can solve the least-squares problem

$$\min \sum_{k=1}^{n} (I_k - I_{\text{des}})^2$$

ignoring the constraints. If the solution is not feasible, we saturate it, i.e., set $p_j := 0$ if $p_j \le 0$ and $p_j := 1$ if $p_j \ge 1$. Complete the notebook and compute a feasible p using this method, and calculate $\phi(p)$.

ii *Weighted least-squares.* We consider another least-squares problem:

$$\min \sum_{k=1}^{n} (I_k - I_{\text{des}})^2 + \mu \sum_{i=1}^{m} (p_i - 0.5)^2,$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

where $\mu \geq 0$ is used to attach a cost to a deviation of the powers from the value 0.5, which lies in the middle of the power limits. For large enough μ, the solution of this problem will satisfy $0 \leq p_i \leq 1$, i.e., be feasible for the original problem. Complete the notebook and find the smallest μ such that p becomes feasible, and evaluate $\phi(p)$.

(c) Complete the notebook. Solve the LP you derived in part (a). Compare the solution with the solutions you obtained using the (weighted) least-squares methods of part (b).

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

5. Learning a Piecewise Constant Function

A piecewise constant function is one that remains constant on intervals and may change values at specific points called **kinks**. Formally, it can be written as:

$$f(x) = \begin{cases} c_1, & \text{if } x \in [a_0, a_1), \\ c_2, & \text{if } x \in [a_1, a_2), \\ \vdots & \vdots \\ c_k, & \text{if } x \in [a_{k-1}, a_k), \end{cases}$$

where c_i are constants and a_i are the kinks.

We are given n noisy observations $\{(x_i, y_i)\}_{i=1}^n$, with $x_i, y_i \in \mathbb{R}$ and

$$y_i = f(x_i) + \epsilon_i,$$

where ϵ_i represents measurement noise.

Our goal is to learn f by formulating a regularized least squares problem. Let $\vec{y} = [y_1, y_2, \ldots, y_n]^\top \in \mathbb{R}^n$ denote the vector of our noisy observations, and let $\hat{\vec{y}} = [\hat{f}(x_1), \hat{f}(x_2), \ldots, \hat{f}(x_n)]^\top \in \mathbb{R}^n$ represent the vector of our estimated function values. Because f is expected to have far fewer kinks than the number of observations n, we add an ℓ_1 regularization term that enforces sparsity in these kinks. This term is combined with the usual least squares data-fidelity objective, $\|\vec{y} - \hat{\vec{y}}\|_2^2$.

To detect the kinks, we first sort the pairs (x_i, y_i) in ascending order of x_i. We then exploit the fact that the difference between y_i and y_{i+1} is small unless x_i lies near a kink.

(a) Formulate the regularized least squares problem using $\lambda > 0$ as the regularization parameter.

(b) Implement your formulation in the notebook.

(c) Change the values of λ and visualize the corresponding learned function. Describe your observation.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

6. A review of standard problem formulations

In this question, we review conceptually the standard forms of various problems and the assertions we can (and cannot!) make about each. The solutions are given, you just need to read them, understand them, and copy-paste them to your submission.

(a) ***Linear programming (LP).***

i. Write the most general form of a linear program (LP) and list its defining attributes.

Solution: A general LP can be written as

$$p^* = \min_{\vec{x} \in \mathbb{R}^n} \quad \vec{c}^\top \vec{x} + d \tag{4}$$

$$\text{s.t.} \quad A_{\text{eq}} \vec{x} = \vec{b}_{\text{eq}} \tag{5}$$

$$A\vec{x} \leq \vec{b}, \tag{6}$$

or equivalently,

$$p^* = \min_{\vec{x} \in \mathbb{R}^n} \quad \vec{c}^\top \vec{x} + d \tag{7}$$

$$\text{s.t.} \quad A_{\text{eq}} \vec{x} = \vec{b}_{\text{eq}} \tag{8}$$

$$\vec{x} \geq \vec{0}. \tag{9}$$

The first LP formulation is known as the *inequality form*; the second is known as the *conic form*. A full treatment of the equivalence of these forms and how to convert between them can be found in section 9.3 of Calafiore & El Ghaoui.

ii. Under what conditions is an LP convex?

Solution: An LP is **always convex**, as the objective function and all constraints are convex (affine) and all equality constraints are affine.

(b) ***Quadratic programming (QP).***

i. Write the most general form of a quadratic program (QP) and list its defining attributes.

Solution: A general QP can be written as

$$p^* = \min_{\vec{x} \in \mathbb{R}^n} \quad \frac{1}{2} \vec{x}^\top H \vec{x} + \vec{c}^\top \vec{x} + d \tag{10}$$

$$\text{s.t.} \quad A_{\text{eq}} \vec{x} = \vec{b}_{\text{eq}} \tag{11}$$

$$A\vec{x} \leq \vec{b}. \tag{12}$$

ii. Under what conditions is a QP convex?

Solution:A QP is convex if and only if $H \succeq 0$ (i.e., PSD).

(c) ***Quadratically-constrained quadratic programming (QCQP).***

i. Write the most general form of a quadratically-constrained quadratic program (QCQP) and list its defining attributes.

Solution: A general QCQP can be written as

$$p^* = \min_{\vec{x} \in \mathbb{R}^n} \quad \vec{x}^\top H_0 \vec{x} + 2\vec{c}^\top \vec{x} + d \tag{13}$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

$$\text{s.t.} \quad \vec{x}^\top H_i \vec{x} + 2\vec{c}_i^\top \vec{x} + d_i \leq 0, \quad i = 1, \ldots, m \tag{14}$$

$$\vec{x}^\top H_j \vec{x} + 2\vec{c}_j^\top \vec{x} + d_j = 0, \quad j = 1, \ldots, q. \tag{15}$$

ii. Under what conditions is a QCQP convex?

Solution:A QCQP is convex if and only if all matrices H_0 and H_i, $i = 1, \ldots, m$ are PSD, and $H_j = 0$ for all $j = 1, \ldots, q$ (i.e., when the objective and all inequality constraints are convex quadratic, and all the equality constraints are actually affine.

(d) ***Second-order cone programming (SOCP).***

i. Write the most general form of a second-order cone program (SOCP) and list its defining attributes.

Solution: A general SOCP can be written as

$$p^* = \min_{\vec{x} \in \mathbb{R}^n} \quad \vec{c}^\top \vec{x} \tag{16}$$

$$\text{s.t.} \quad \left\| A_i \vec{x} + \vec{b}_i \right\|_2 \leq \vec{c}_i^\top \vec{x} + d_i, \quad i = 1, \ldots, m, \tag{17}$$

or equivalently,

$$p^* = \min_{\vec{x} \in \mathbb{R}^n} \quad \vec{c}^\top \vec{x} \tag{18}$$

$$\text{s.t.} \quad (A_i \vec{x} + \vec{b}_i, \vec{c}_i^\top \vec{x} + d_i) \in \mathcal{K}_{m_i} \quad i = 1, \ldots, m, \tag{19}$$

where second-order cone (SOC) $\mathcal{K}_n \doteq \{(\vec{x}, t), \vec{x} \in \mathbb{R}^n, t \in \mathbb{R} \mid \|\vec{x}\|_2 \leq t\}$. The first SOCP formulation is known as the *standard inequality form*; the second is known as the *conic standard form*.

ii. Under what conditions is an SOCP convex?

Solution: An SOCP is **always convex**, as the objective function is convex (linear) and the constraint stipulates that points lie within a convex set. A thorough discussion of the convexity of SOC constraints can be found in Calafiore & El Ghaoui chapter 10.

(e) ***Relationships.*** Recall that

$$\text{LP} \subset \text{QP}_{\text{convex}} \subset \text{QCQP}_{\text{convex}} \subset \text{SOCP} \subset \{\text{all convex programs}\}, \tag{20}$$

where LP denotes the set of all linear programs, $\text{QP}_{\text{convex}}$ denotes the set of all convex quadratic programs, etc. Which of these problems can be solved most efficiently? Why are these categorizations useful?

Solution: In general, problems to the left side of the subset sequence above can be solved more efficiently than those on the right, for problems of comparable size: LPs are arguably easiest (optima are always achieved at a critical point, so we just need to check those analytically, e.g. using the simplex algorithm), while general convex problems must use numerical algorithms like gradient descent and Newton's method, whose efficiency depends on the particular geometric characteristics of the problem. Intermediate forms (convex QPs/QCQPs, SOCPs) fall somewhere in between in terms of difficulty: there's lots of research on how to solve them efficiently (one popular set of approaches is called "interior point methods"), and there are a number of off-the-shelf solvers available (e.g., CVX).

Though we don't get a chance to explore it much in this class, knowing these classes of problems is useful when encountering optimization problems in the wild — if you can write your problem in one of the forms above, you know what kinds of solutions and convergence guarantees you can expect, and often employ

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

existing software to help you solve it. For further discussion of these problem classes — as well as an even more general class known as semidefinite programming (SDP), of which SOCPs are a subset — we encourage you to take EECS 227B and 227C.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

7. Homework Process

With whom did you work on this homework? List the names and SIDs of your group members.

NOTE: If you didn't work with anyone, you can put "none" as your answer.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.